

Numerical Experiments of Tensor Over-parametrization (Simulated Version)

Real-world Application

To simulate the effect of tensor over-parameterization in escaping spurious local minima that arise in the matrix space, we consider the following instance of the Matrix Sensing (MS) problem:

Let the ground truth matrix be

$$M^* = zz^\top = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix},$$

where $z = [1, 0, 0]^\top$. The linear measurement operator $\mathcal{A} : \mathbb{R}^{3 \times 3} \rightarrow \mathbb{R}^6$ is defined through the sensing matrices:

$$\begin{aligned} A_1 &= \begin{bmatrix} 0.0783 & 0.2372 & -0.0439 \\ 0.2372 & -0.0397 & 0.1456 \\ -0.0439 & 0.1456 & -0.4724 \end{bmatrix}, & A_2 &= \begin{bmatrix} 0.0389 & 0.0536 & -0.0059 \\ 0.0536 & 0.4614 & -0.3907 \\ -0.0059 & -0.3907 & 0.2760 \end{bmatrix}, \\ A_3 &= \begin{bmatrix} 0.0293 & -0.1456 & -0.1656 \\ -0.1456 & 0.3257 & 0.2528 \\ -0.1656 & 0.2528 & -0.0078 \end{bmatrix}, & A_4 &= \begin{bmatrix} 0.0762 & 0.3193 & -0.3338 \\ 0.3193 & -0.3364 & 0.5847 \\ -0.3338 & 0.5847 & 0.1873 \end{bmatrix}, \\ A_5 &= \begin{bmatrix} -0.0889 & 0.7089 & 0.4472 \\ 0.7089 & 0.3788 & 0.0902 \\ 0.4472 & 0.0902 & -0.3193 \end{bmatrix}, & A_6 &= \begin{bmatrix} 0.4097 & 0.1190 & 0.2078 \\ 0.1190 & 0.2282 & -0.2274 \\ 0.2078 & -0.2274 & 0.4046 \end{bmatrix}. \end{aligned}$$

We consider the following objective function:

$$h(x) = f(xx^\top) = \frac{1}{2} \|\mathcal{A}(xx^\top - zz^\top)\|_2^2,$$

where $x \in \mathbb{R}^3$ is the optimization variable (we still use $X \in \mathbb{R}^{3 \times 1}$ later for consistency). The operator $\mathcal{A}$ satisfies the Restricted Isometry Property (RIP) of order 2 with constant ≈ 0.875 .

Let

$$\hat{x} = [0.2234 \quad 0.0918 \quad 0.5985]^\top.$$

As shown in the manuscript, $\hat{x}$ is a spurious local minimum of the objective $h(x)$.

```
In [1]: %matplotlib inline
import matplotlib_inline # setup output image format, do not use 'svg' if need 'pdf'
matplotlib_inline.backend_inline.set_matplotlib_formats('svg')
import matplotlib.pyplot as plt
plt.rcParams['figure.dpi'] = 300
```

```

import matplotlib
import argparse
import numpy as np
import jax
import jax.numpy as jnp
from typing import List, Tuple
from itertools import chain
from tqdm import tqdm

```

Functional Helpers for Matrix Computation

- Apply the linear measurement operator $\mathcal{A}$ (*func*: `apply_A`):

$$\mathcal{A}(M) = [\langle A_1, M \rangle, \langle A_2, M \rangle, \dots, \langle A_m, M \rangle]^\top \in \mathbb{R}^m.$$

- Apply the adjoint operator $\mathcal{A}^*$ (*func*: `adjoint_A`):

$$\mathcal{A}^*(y) = \sum_{i=1}^m y_i \cdot A_i \in \mathbb{R}^{n \times n}.$$

- Compute the smallest eigenvalue λ_n and eigenvector u_n of a symmetric matrix G (*func*: `min_eigpair_symmetric`):

$$G = \sum_{\theta=1}^n \lambda_\theta u_\theta u_\theta^\top, \quad \lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n.$$

- Compute the smallest nonzero singular value σ_r and associated vectors v_r and q_r of X (*func*: `last_nonzero_singular_triplet`):

$$X = \sum_{\phi=1}^r \sigma_\phi v_\phi q_\phi^\top, \quad \sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0.$$

```

In [2]: def apply_A(A, M):
        return np.einsum("ijk,jk->i", A, M)

def adjoint_A(A, y):
    weighted_A = y[:, np.newaxis, np.newaxis] * A
    G = np.sum(weighted_A, axis=0)
    return 0.5 * (G + G.T)

def min_eigpair_symmetric(G):
    vals, vecs = np.linalg.eigh(G)
    idx = np.argmin(vals)
    return float(vals[idx]), vecs[:, idx]

def last_nonzero_singular_triplet(X, rtol=1e-10):
    """
    Computes the last non-zero singular value ( $\sigma_r$ ) of a matrix X,
    along with its corresponding left and right singular vectors ( $v_r$ ,  $q_r$ ).

    Parameters:
        X (np.ndarray): Input matrix of shape (n, r). Can also be a vector (1D or 2D).
        rtol (float): Relative tolerance to determine whether a singular value is zero.

    Returns:
        sigma_r (float): The smallest non-zero singular value ( $\sigma_r$ ).
        v_r (np.ndarray): Corresponding left singular vector (shape: (n,)).
    """

```

```

q_r (np.ndarray): Corresponding right singular vector (shape: (r,)).

Raises:
    ValueError: If the matrix has rank 0 (i.e., all singular values are zero).
"""
# Ensure X is at least 2D (e.g., convert a vector to column matrix if needed)
X = np.atleast_2d(X)

# Perform singular value decomposition: X = V S Q^T
V, S, Qh = np.linalg.svd(X, full_matrices=False)

# Determine the numerical rank by counting singular values > tolerance
tol = rtol * S[0] # tolerance is relative to the largest singular value
rank = np.sum(S > tol)

if rank == 0:
    raise ValueError("The input matrix has rank 0 - no non-zero singular values.")

# Get the last non-zero singular value and its corresponding singular vectors
sigma_r = S[rank - 1] # Last non-zero singular value
v_r = V[:, rank - 1] # Corresponding left singular vector
q_r = Qh[rank - 1, :] # Corresponding right singular vector (from Q^T)

return sigma_r, v_r, q_r

```

Functional Helpers for Matrix Sensing and Optimization

- Evaluate the objective function $h(X)$ in matrix space (*func*: `matrix_objective`):

$$h(X) = \frac{1}{2} \|\mathcal{A}(XX^\top - ZZ^\top)\|_2^2 = \frac{1}{2} \langle \mathcal{A}(XX^\top) - b, \mathcal{A}(XX^\top) - b \rangle,$$

where $b = \mathcal{A}(ZZ^\top) = \mathcal{A}(M^*) \in \mathbb{R}^m$.

- Compute the gradient $\nabla h(X)$ (*func*: `matrix_gradient`):

$$\nabla h(X) = 2\mathcal{A}^* \mathcal{A}(XX^\top - ZZ^\top)X = 2 \sum_{i=1}^m \langle A_i, XX^\top - ZZ^\top \rangle A_i X = 2 \sum_{i=1}^m (\langle A_i, XX^\top \rangle - b_i) A_i X = 2GX \in \mathbb{R}^{n \times r},$$

where the symmetric matrix $G \in \mathbb{R}^{n \times n}$ is defined as:

$$G = \mathcal{A}^* \mathcal{A}(XX^\top - ZZ^\top) = \sum_{i=1}^m \langle A_i, XX^\top - ZZ^\top \rangle A_i = \sum_{i=1}^m (\langle A_i, XX^\top \rangle - b_i) A_i.$$

- Run vanilla gradient descent (GD) in matrix space (*func*: `run_gd`):

$$X^{(t+1)} = X^{(t)} - \text{lr} \cdot \nabla h(X^{(t)}).$$

- Estimate the RIP constant δ_p via random sampling (*func*: `estimate_delta_p`):

$$(1 - \delta_p) \|XX^\top\|_F^2 \leq \|\mathcal{A}(XX^\top)\|_2^2 \leq (1 + \delta_p) \|XX^\top\|_F^2.$$

In [3]: `def matrix_objective(A, X, b):`
`M = X @ X.T`

```

y = apply_A(A, M)
r = y - b
return 0.5 * float(r @ r)

def matrix_gradient(A, X, b):
    M = X @ X.T
    y = apply_A(A, M)
    G_M = adjoint_A(A, y - b)
    return 2.0 * (G_M @ X)

def run_gd(A, Z, b, X0, lr, steps, grad_tol):
    X = X0.copy()
    traj = [X.copy()]
    losses = [matrix_objective(A, X, b)]
    gradnorms = [float(np.linalg.norm(matrix_gradient(A, X, b), "fro"))]
    dists = [np.linalg.norm(X @ X.T - Z @ Z.T, "fro")]

    for _ in range(steps):
        nabla_h = matrix_gradient(A, X, b)
        X = X - lr * nabla_h

        gnorm = float(np.linalg.norm(matrix_gradient(A, X, b), "fro"))
        traj.append(X.copy())
        losses.append(matrix_objective(A, X, b))
        gradnorms.append(gnorm)
        dists.append(np.linalg.norm(X @ X.T - Z @ Z.T, "fro"))
        if gnorm <= grad_tol:
            break

    return traj, np.array(losses), np.array(gradnorms), np.array(dists)

def estimate_delta_p(A, p, num_samples=5000, seed=0, matrix_type='symmetric', normalize=
"""
Estimate RIP constant delta_p for a linear operator A.

Parameters:
-----
A : np.ndarray
    Array of shape (m, n, n), representing m sensing matrices A_i of size n x n.
p : int
    Target rank for low-rank matrix sampling.
num_samples : int
    Number of random samples to estimate the RIP constant.
seed : int
    Random seed for reproducibility.
matrix_type : str
    Type of matrix to sample: 'general', 'symmetric', or 'psd'.
normalize : bool
    Whether to normalize sampled matrices to unit Frobenius norm.

Returns:
-----
delta_p : float
    Estimated RIP constant.
"""
rng = np.random.default_rng(seed)
m, n, _ = A.shape
A_flat = A.reshape(m, -1) # shape (m, n^2)

r_min, r_max = float("inf"), float("-inf")

for _ in range(num_samples):
    # --- Step 1: Sample a rank-p matrix M ---
    if matrix_type == 'psd':

```

```

X = rng.standard_normal((n, p))
M = X @ X.T # PSD
elif matrix_type == 'symmetric':
    X = rng.standard_normal((n, p))
    M = X @ rng.standard_normal((p, n))
    M = 0.5 * (M + M.T) # Symmetric
elif matrix_type == 'general':
    X = rng.standard_normal((n, p))
    Y = rng.standard_normal((n, p))
    M = X @ Y.T # General rank-p
else:
    raise ValueError(f"Invalid matrix_type: {matrix_type}")

# --- Step 2: Normalize if needed ---
fro_norm = np.linalg.norm(M, 'fro')
if fro_norm == 0:
    continue
if normalize:
    M = M / fro_norm

# --- Step 3: Compute measurement: y = A(M) ---
M_flat = M.reshape(-1) # shape (n^2,)
y = A_flat @ M_flat # shape (m,)

# --- Step 4: Compute squared measurement norm ---
val = y @ y
if normalize:
    val = val # Already normalized
else:
    val = val / (fro_norm ** 2) # adjust for unnormalized M

# --- Step 5: Track min and max ---
r_min = min(r_min, val)
r_max = max(r_max, val)

# --- Step 6: Estimate delta_p ---
delta_p = max(1.0 - r_min, r_max - 1.0)
return float(delta_p)

```

Functional Helpers for Tensor Lifting

- Lift a vector $\text{vec}(X)$ to tensor space (*func*: `elevate_vector`):

$$\mathbf{w} = \text{vec}(X)^{\otimes l} \in \mathbb{R}^{(nr)^{\circ l}}, \quad \text{where } X \in \mathbb{R}^{n \times r}.$$

- Evaluate the lifted objective $h^l(\mathbf{w})$ in tensor space (*func*: `tensor_objective`):

$$h^l(\mathbf{w}) = \frac{1}{2} \|\langle \mathbf{A}^{\otimes l}, \langle \mathbf{P}(\mathbf{w}), \mathbf{P}(\mathbf{w}) \rangle_{2,4,\dots,2l} \rangle_{2,3,5,6,\dots,3l-1,3l} - b^{\otimes l} \|_F^2.$$

Equivalently,

$$h^l(\mathbf{w}) = \frac{1}{2} \|\langle \mathbf{A}^{\otimes l}, \langle X^{\otimes l}, X^{\otimes l} \rangle_{2,4,\dots,2l} - \langle Z^{\otimes l}, Z^{\otimes l} \rangle_{2,4,\dots,2l} \rangle_{2,3,5,6,\dots,3l-1,3l} \|_F^2.$$

To simplify notation, we omit contraction subscripts and expand the inner product

$$\langle \langle \mathbf{A}^{\otimes l}, \langle X^{\otimes l}, X^{\otimes l} \rangle - \langle Z^{\otimes l}, Z^{\otimes l} \rangle \rangle, \langle \mathbf{A}^{\otimes l}, \langle X^{\otimes l}, X^{\otimes l} \rangle - \langle Z^{\otimes l}, Z^{\otimes l} \rangle \rangle,$$

yielding

$$\begin{aligned}
&= \underbrace{\langle\langle \mathbf{A}^{\otimes l}, \langle X^{\otimes l}, X^{\otimes l} \rangle \rangle, \langle \mathbf{A}^{\otimes l}, \langle X^{\otimes l}, X^{\otimes l} \rangle \rangle\rangle}_{:=A_{www}} + \underbrace{\langle\langle \mathbf{A}^{\otimes l}, \langle Z^{\otimes l}, Z^{\otimes l} \rangle \rangle, \langle \mathbf{A}^{\otimes l}, \langle Z^{\otimes l}, Z^{\otimes l} \rangle \rangle\rangle}_{:=A_{zzzz}} \\
&\quad - 2 \underbrace{\langle\langle \mathbf{A}^{\otimes l}, \langle Z^{\otimes l}, Z^{\otimes l} \rangle \rangle, \langle \mathbf{A}^{\otimes l}, \langle X^{\otimes l}, X^{\otimes l} \rangle \rangle\rangle}_{:=A_{zzww}}.
\end{aligned}$$

A commonly reused term `A_topA` is precomputed as:

$$\mathbf{A}^* \mathbf{A} := \langle \mathbf{A}, \mathbf{A} \rangle_1 \in \mathbb{R}^{n \times n \times n \times n},$$

where $\mathbf{A}[i, :, :] = A_i \in \mathbb{R}^{n \times n}$.

- Precompute lifted ground truth term (*func*: `compute_Azzzz`):

$$\begin{aligned}
A_{zzww} &= \langle\langle \mathbf{A}^{\otimes l}, \langle Z^{\otimes l}, Z^{\otimes l} \rangle \rangle, \langle \mathbf{A}^{\otimes l}, \langle X^{\otimes l}, X^{\otimes l} \rangle \rangle \rangle \\
&= \langle\langle \mathbf{A}^{\otimes l}, \langle X^{\otimes l}, X^{\otimes l} \rangle \rangle, \langle \mathbf{A}^{\otimes l}, \langle Z^{\otimes l}, Z^{\otimes l} \rangle \rangle \rangle = A_{wwzz},
\end{aligned}$$

which can be reused to avoid redundant computation in the lifted loss.

- Approximate the lifted objective (*func*: `tensor_objective_approx`):

Given a parametric approximation of the lifted tensor

$$\mathbf{w} \approx \sum_j c_j \cdot x_j^{\otimes l},$$

we define the approximate tensor objective as:

$$\tilde{h}^l(\mathbf{w}) = \left\| \sum_{j,k} c_j c_k \cdot (x_j^\top A_i x_k)^l - (z^\top A_i z)^l \right\|_2^2, \quad \text{where } \text{rank}(M^*) = 1.$$

This avoids constructing the full lifted tensor $\mathbf{w} \in \mathbb{R}^{(nr)^{ol}}$ and accelerates computing.

```
In [4]: def elevate_vector(w_in, level):
        """
        Elevates a 1D vector to a higher-dimensional tensor space.

        Parameters:
        -----
        w_in : array-like
            Input vector of shape (n,).
        level : int
            The lifting level; the final tensor has dimensionality (odd number).

        Returns:
        -----
        lifted : array-like
            A high-dimensional lifted tensor.
        """
        if level == 0:
            return w_in
        w_in = w_in.reshape(-1) # Flatten the input to ensure 1D
        einsum_w_args = list(chain(*[(w_in, (i,)) for i in range(level)]))
        lifted_tensor = jnp.einsum(*einsum_w_args) # Perform Einstein summation
        return lifted_tensor

def tensor_objective(w, z_lifted, A_topA, Azzzz, level):
    """
```

Computes the loss in a memory-efficient way.

Parameters:

w : jnp.ndarray
 Current variable in the lifted space.
z_lifted : jnp.ndarray
 Lifted version of the target vector z.
A_topA : jnp.ndarray
 Precomputed tensor product of A with itself.
Azzzz : jnp.ndarray
 Precomputed value of the lifted tensor z.
level : int
 The lifting level.

Returns:

loss : float
 The computed loss.
"""
Build arguments for einsum
args1 = []
for i in range(level):
 args1 += [A_topA, (i * 4, i * 4 + 1, i * 4 + 2, i * 4 + 3)]

args2 = [w, tuple(np.arange(level) * 4)]
args3 = [w, tuple(np.arange(level) * 4 + 1)]
args4 = [w, tuple(np.arange(level) * 4 + 2)]
args5 = [w, tuple(np.arange(level) * 4 + 3)]

args = args1 + args2 + args3 + args4 + args5

Compute Awww
Awww = jnp.einsum(*args)

Rebuild args for Azzww
args2 = [z_lifted, tuple(np.arange(level) * 4)]
args3 = [z_lifted, tuple(np.arange(level) * 4 + 1)]

args = args1 + args2 + args3 + args4 + args5

Azzww = jnp.einsum(*args)

Compute loss
loss = 0.5 * (Awww + Azzzz - 2 * Azzww)
return loss

def compute_Azzzz(A_topA, z_lifted, level):

"""

Computes the lifted tensor Azzzz in a higher-dimensional space.

Parameters:

A_topA : jnp.ndarray
 Tensor of shape (n, n, n, n).
z_lifted : jnp.ndarray
 Lifted tensor of shape (level,).
level : int
 The lifting level (must be odd).

Returns:

Azzzz : jnp.ndarray
 The lifted tensor in the higher-dimensional space.

```

"""
# Compute args1
args1 = []
for i in range(level):
    args1 += [A_topA, (i * 4, i * 4 + 1, i * 4 + 2, i * 4 + 3)]

# Compute args2, args3, args4, args5
args2 = [z_lifted, tuple(np.arange(level) * 4)]
args3 = [z_lifted, tuple(np.arange(level) * 4 + 1)]
args4 = [z_lifted, tuple(np.arange(level) * 4 + 2)]
args5 = [z_lifted, tuple(np.arange(level) * 4 + 3)]

# Combine args and compute Azzzz
args = args1 + args2 + args3 + args4 + args5
return jnp.einsum(*args)

def tensor_objective_approx(
    A: np.ndarray,
    terms: List[Tuple[float, np.ndarray]],
    Z: np.ndarray,
    level: int
) -> float:
    """
    Computes a fast, tensor-free approximation of the lifted tensor objective.

    Given a set of measurement matrices  $A_i$  and a list of weighted vectors
    approximating a low-rank tensor  $w \approx \sum_i c_i * x_i^{\{\otimes\}}$ , this function
    evaluates the objective:

        
$$\sum_i (y_i - b_i)^2$$


    where:
        -  $y_i = \sum_{j,k} c_j * c_k * (x_j^T A_i x_k)^l$ 
        -  $b_i = (z^T A_i z)^l$ 

    This is a fast approximation of the full tensor objective without explicitly
    building the lifted tensor or  $A^{\{\otimes\}}$ .

    Parameters
    -----
    A : np.ndarray of shape (m, n, n)
        A stack of m symmetric measurement matrices  $A_i \in \mathbb{R}^{n \times n}$ .

    terms : list of tuples (c_j, x_j)
        Each x_j is a vector of shape (n,) and c_j is a scalar coefficient.
        The list approximates the lifted tensor as  $\sum_j c_j * x_j^{\{\otimes\}}$ .

    Z : np.ndarray of shape (n,)
        The target vector.

    level : int
        The lifting level l (i.e., the tensor order for Kronecker powers).

    Returns
    -----
    float
        The lifted objective value:
        
$$||\text{lifted\_measurements} - \text{ground\_truth}||^2$$

    """
    m, n, n_check = A.shape
    assert n == n_check, "A must be of shape (m, n, n)"

    # Flatten input vectors
    xs = [x.reshape(-1) for _, x in terms]

```

```

cs = [float(c) for c, _ in terms]

# Compute the lifted measurements y_i
y = np.zeros(m, dtype=np.float64)
for i in range(m):
    Ai = A[i]
    acc = 0.0
    for j, (cj, xj) in enumerate(zip(cs, xs)):
        for k, (ck, xk) in enumerate(zip(cs, xs)):
            dot = float(xj.T @ (Ai @ xk)) # Bilinear form
            acc += cj * ck * (dot ** level)
    y[i] = acc

# Compute the ground truth lifted measurements (z^T A_i z)^l
Z = Z.reshape(-1)
b = np.array([float(Z.T @ (A[i] @ Z)) ** level for i in range(m)], dtype=np.float64)

# Return the objective value: ||y - b||^2
return np.sum((y - b) ** 2)

```

Load Hyper-parameters and MS Problem

```

In [5]: # Load hyper-parameters
class Args:
    seed = 0 # Random seed
    r = 2 # Rank parameter r for RIP sampling (delta_2r*=1/2 is a sharp bound)
    delta_samples = 5000 # Number of RIP samples
    l = 5 # Lift order l
    rho = 0.1 # Escape step size in tensor space
    eta = 0.1 # Truncated PGD step size in tensor space
    t = 150000 # Truncated PGD step numbers (Simulated)
    init_eps = 1.0 # Init magnitude epsilon
    gd_lr = 0.1 # Matrix-space GD learning rate
    gd_steps_before_lift = 100 # GD steps before lifting
    gd_steps_after_collapse = 100 # GD steps after collapsing
    gd_gradtol = 1e-8 # Stopping threshold for ||grad||_F
args = Args()

# Load MS problem
def load_problem():
    n, m, r = 3, 6, 1
    A1 = np.array([[0.0783, 0.2372, -0.0439],
                   [0.2372, -0.0397, 0.1456],
                   [-0.0439, 0.1456, -0.4724]])
    A2 = np.array([[0.0389, 0.0536, -0.0059],
                   [0.0536, 0.4614, -0.3907],
                   [-0.0059, -0.3907, 0.2760]])
    A3 = np.array([[0.0293, -0.1456, -0.1656],
                   [-0.1456, 0.3257, 0.2528],
                   [-0.1656, 0.2528, -0.0078]])
    A4 = np.array([[0.0762, 0.3193, -0.3338],
                   [0.3193, -0.3364, 0.5847],
                   [-0.3338, 0.5847, 0.1873]])
    A5 = np.array([[ -0.0889, 0.7089, 0.4472],
                   [0.7089, 0.3788, 0.0902],
                   [0.4472, 0.0902, -0.3193]])
    A6 = np.array([[0.4097, 0.1190, 0.2078],
                   [0.1190, 0.2282, -0.2274],
                   [0.2078, -0.2274, 0.4046]])
    A = np.vstack([A1, A2, A3, A4, A5, A6]).reshape((m, n, n))
    Z = np.array([[1.0], [0.0], [0.0]])
    X_hat = np.array([[0.2234], [0.0918], [0.5985]])
    return n, m, r, A, Z, X_hat

```

```

n, m, r, A, Z, X_hat = load_problem()
Z = Z.astype(float)
M_star = Z @ Z.T
X_hat = X_hat.astype(float)
M_hat = X_hat @ X_hat.T

```

Compute the Escape Feasibility Score (EFS)

The Escape Feasibility Score is defined as:

$$\text{EFS} := \underbrace{\frac{-\lambda_n}{\sigma_r^2(1 + \delta_p)}}_{:=\text{CSR}} + \underbrace{\frac{(\text{daa_tmp})^2}{2(1 + \delta_p)^2}}_{:=\text{DAA}},$$

where

$$\text{daa_tmp} := \sum_{i=1}^m \langle A_i, u_n v_r^\top + v_r u_n^\top \rangle \langle A_i, u_n u_n^\top \rangle.$$

If $\text{EFS} > 1$, the current point $\hat{x}$ is escapeable via a single-step perturbation.

```

In [6]: b = apply_A(A, M_star)
        residual = apply_A(A, M_hat) - b
        G = adjoint_A(A, residual)
        lambda_min, u_n = min_eigpair_symmetric(G)
        sigma_r, v_r, q_r = last_nonzero_singular_triplet(X_hat)
        uvvu = np.outer(u_n, v_r) + np.outer(v_r, u_n)
        uu = np.outer(u_n, u_n)
        daa_tmp = np.sum(
            np.sum(A * uvvu, axis=(1, 2)) * np.sum(A * uu, axis=(1, 2))
        )
        delta_p = estimate_delta_p(A, p=args.r, num_samples=args.delta_samples, seed=args.seed,
                                   matrix_type='symmetric', normalize=True)
        CSR = (-lambda_min) / (sigma_r ** 2 * (1 + delta_p))
        DAA = (daa_tmp ** 2) / (2 * (1 + delta_p) ** 2)
        EFS = CSR + DAA

        print("==== Single Step Escape =====")
        print(f"n={n}, m={m}, r={r}, delta_p={delta_p:.6f}")
        print(f"lambda_min={lambda_min:.6f}, sigma_r={sigma_r:.6f}, daa_tmp={daa_tmp:.6f}")
        print(f"CSR={CSR:.6f}, DAA={DAA:.6f}, EFS={EFS:.6f}")

```

```

==== Single Step Escape =====
n=3, m=6, r=1, delta_p=0.875287
lambda_min=-0.133233, sigma_r=0.645397, daa_tmp=0.633380
CSR=0.170565, DAA=0.057038, EFS=0.227603

```

Run GD in Matrix Space (Pre-Escape Phase)

```

In [7]: rng = np.random.default_rng(args.seed)
        X0 = X_hat + args.init_eps * rng.normal(size=X_hat.shape)
        traj_before, losses_before, gradnorms_before, dists_before = run_gd(
            A, Z, b, X0, args.gd_lr, args.gd_steps_before_lift, args.gd_gradtol)
        X_local = traj_before[-1]
        print(X_local)
        print(len(traj_before))
        print(len(losses_before))

```

```

print(len(gradnorms_before))
print(len(dists_before))

# Set up a 1x3 grid of subplots
fig1, axes1 = plt.subplots(1, 3, figsize=(10, 3))

ax1A = axes1[0]
ax1A.plot(range(args.gd_steps_before_lift+1), losses_before, color='black', label='loss')
ax1A.set_xlabel('Iteration', fontsize=11)
ax1A.set_ylabel(r'$h(X)$', fontsize=11)
ax1A.tick_params(axis='both', labels=11)
ax1A.set_title('Loss vs Iteration', fontsize=12)
ax1A.legend(frameon=False, fontsize=11)
ax1A.grid(True)

ax1B = axes1[1]
ax1B.plot(range(args.gd_steps_before_lift+1), gradnorms_before, color='black', label='gr')
ax1B.set_xlabel('Iteration', fontsize=11)
ax1B.set_ylabel(r'$||\nabla h(X)||_F$', fontsize=11)
ax1B.tick_params(axis='both', labels=11)
ax1B.set_title('Gradient Norm vs Iteration', fontsize=12)
ax1B.legend(frameon=False, fontsize=11)
ax1B.grid(True)

ax1C = axes1[2]
ax1C.plot(range(args.gd_steps_before_lift+1), dists_before, color='black', label='distan')
ax1C.set_xlabel('Iteration', fontsize=11)
ax1C.set_ylabel(r'$||XX^T - ZZ^T||_F$', fontsize=11)
ax1C.tick_params(axis='both', labels=11)
ax1C.set_title('Distance vs Iteration', fontsize=12)
ax1C.legend(frameon=False, fontsize=11)
ax1C.grid(True)

plt.tight_layout()
plt.show()

```

```

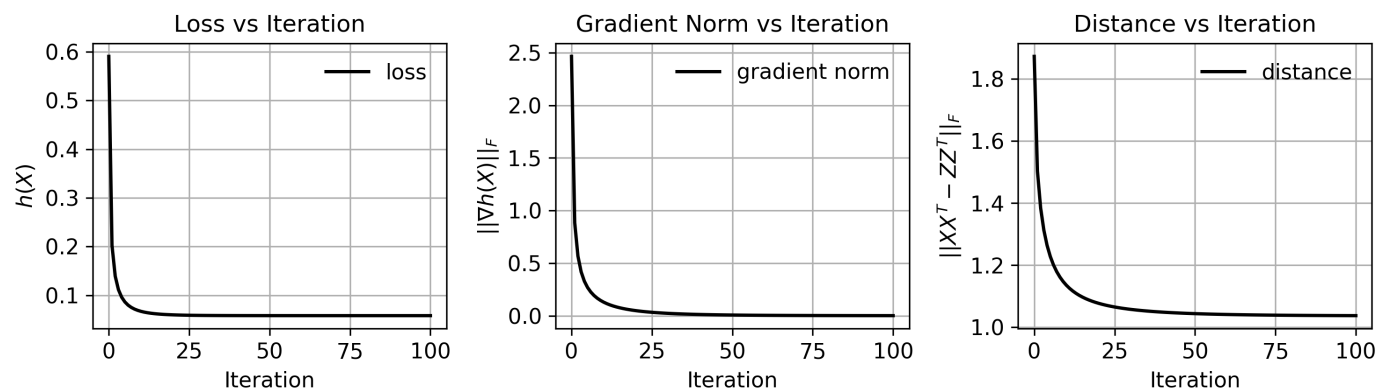
[[0.22238989]
 [0.09220103]
 [0.59963197]]

```

```

101
101
101
101

```



Simulated Tensor Over-parameterization

We consider three base tensors in the lifted subspace $\mathcal{S}$:

$$\begin{aligned} \mathbf{a_vec} &:= \text{vec}(\hat{X}), & \mathbf{a} &:= \text{vec}(\hat{X})^{\otimes l}, \\ \mathbf{b_vec} &:= \text{vec}(u_n q_r^\top), & \mathbf{b} &:= \text{vec}(u_n q_r^\top)^{\otimes l}, \\ \mathbf{c_vec} &:= \text{vec}(\mathcal{A}^* \mathcal{A}(u_n v_r^\top + v_r u_n^\top) \hat{X}), & \mathbf{c} &:= \text{vec}(\mathcal{A}^* \mathcal{A}(u_n v_r^\top + v_r u_n^\top) \hat{X})^{\otimes l}. \end{aligned}$$

We define the perturbed tensor (escape direction) as:

$$\mathbf{w}_{\text{escape}} := \mathbf{a} + \rho \cdot \mathbf{b}.$$

The lifted objective at the saddle point is denoted as:

$$h^l(\hat{\mathbf{w}}) = h^l(\mathbf{a}) =: \text{hl_alpha_a},$$

assuming $\tilde{\alpha}^{(t)} = 1$ during the truncated projected gradient descent (simulated) process.

The geometric sum used in coefficient updates is:

$$\sum_{\tau=0}^t \nu^\tau = \frac{1 - \nu^{t+1}}{1 - \nu}, \quad \text{where } \nu := 1 - \eta \lambda^l.$$

The dynamics of the lifted rank-1 tensors are:

$$\begin{aligned} \tilde{\alpha}^{(t)} &= 1, & \mathbf{a_lift} &= \tilde{\alpha}^{(t)} \cdot \mathbf{a}, & \text{hl_alpha_a} &= h^l(\mathbf{a_lift}), \\ \tilde{\beta}^{(t)} &= \rho \nu^t, & \mathbf{b_lift} &= \tilde{\beta}^{(t)} \cdot \mathbf{b}, & \text{hl_beta_b} &= h^l(\mathbf{b_lift}), \\ \tilde{\gamma}^{(t)} &= -\eta \rho \frac{\sigma_r^l}{2^{l-1}} \left(\sum_{\tau=0}^{t-1} \nu^\tau \right), & \mathbf{c_lift} &= \tilde{\gamma}^{(t)} \cdot \mathbf{c}, & \text{hl_gamma_c} &= h^l(\mathbf{c_lift}). \end{aligned}$$

- If $\|\mathbf{c_lift}\|_F \gg \max\{\|\mathbf{a_lift}\|_F, \|\mathbf{b_lift}\|_F\}$ and $\text{hl_gamma_c} < h^l(\hat{\mathbf{w}})$, then escape is successful via collapsing the $\mathbf{c_lift}$ term.
- If $\|\mathbf{b_lift}\|_F \gg \max\{\|\mathbf{a_lift}\|_F, \|\mathbf{c_lift}\|_F\}$ and $\text{hl_beta_b} < h^l(\hat{\mathbf{w}})$, then escape is successful via collapsing the $\mathbf{b_lift}$ term.

In [8]:

```
l = args.l
a_vec = X_hat.reshape(-1)
b_vec = np.outer(u_n, q_r).reshape(-1)
c_vec = (adjoint_A(A, apply_A(A, uvvu)) @ X_hat).reshape(-1)
lam_pow = lambda_min ** l
nu = 1.0 - args.eta * lam_pow

def geom_sum(t):
    if abs(nu - 1.0) < 1e-12:
        return float(t + 1)
    return float((1.0 - (nu ** (t + 1))) / (1.0 - nu))

# lifted constants
A_topA = jnp.einsum("ijk,ilm->jklm", A, A)
z_lifted = elevate_vector(Z, level=l)
Azzzz = compute_Azzzz(A_topA, z_lifted, level=l)

T = args.t
alpha_t = 1.0 # does not change with t
a_vec_lift = elevate_vector(a_vec, level=l)
b_vec_lift = elevate_vector(b_vec, level=l)
c_vec_lift = elevate_vector(c_vec, level=l)
w_escape = a_vec_lift + args.rho * b_vec_lift
#w_escape_rank1_vec = a_vec + (args.rho ** (1 / l) / sigma_r) * b_vec # rank-1 approx i
```

```

a_lift = alpha_t * a_vec_lift
hl_alpha_a = tensor_objective(a_vec_lift, z_lifted, A_topA, Azzzz, level=1)
hl_alpha_a_approx = tensor_objective_approx(A, [(alpha_t, a_vec)], Z, level=1)
hl_w_escape = tensor_objective(w_escape, z_lifted, A_topA, Azzzz, level=1)
#hl_w_escape_approx = tensor_objective_approx(A, [(1.0, w_escape_rank1_vec)], Z, level=1)
hl_w_escape_approx = tensor_objective_approx(A, [(alpha_t, a_vec), (args.rho, b_vec)], Z
print("Tensor Objective of Saddle Point: ", hl_alpha_a)
print("Approximated Version of h(hat_w): ", hl_alpha_a_approx)
print("Tensor Objective of Saddle Point towards Escape Direction with Single-step: ", hl
print("Approximated Version of h(w_escape): ", hl_w_escape_approx)

```

```

Tensor Objective of Saddle Point: 0.00012294958
Approximated Version of h(hat_w): 0.00012546038397646264
Tensor Objective of Saddle Point towards Escape Direction with Single-step: 0.000126147
48
Approximated Version of h(w_escape): 0.00012626580585702927

```

In [9]:

```

#hl_beta_b_list = []
#hl_gamma_c_list = []
hl_w_list = []
a_lift_list = []
b_lift_list = []
c_lift_list = []

for tt in tqdm(range(T)):
    beta_t = args.rho * (nu ** (tt + 1))
    gamma_t = -args.eta * args.rho * ((sigma_r ** 1) / (2 ** (1 - 1))) * geom_sum(tt)
    b_lift = beta_t * b_vec_lift
    c_lift = gamma_t * c_vec_lift

    hl_beta_b_approx = tensor_objective_approx(A, [(beta_t, b_vec)], Z, level=1)
    hl_gamma_c_approx = tensor_objective_approx(A, [(gamma_t, c_vec)], Z, level=1)
    hl_w_t_approx = tensor_objective_approx(A, [(alpha_t, a_vec),
                                                (beta_t, b_vec),
                                                (gamma_t, c_vec)], Z, level=1)

    #hl_beta_b_list.append(hl_beta_b_approx)
    #hl_gamma_c_list.append(hl_gamma_c_approx)
    hl_w_list.append(hl_w_t_approx)

    a_lift_norm = jnp.linalg.norm(a_lift)
    b_lift_norm = jnp.linalg.norm(b_lift)
    c_lift_norm = jnp.linalg.norm(c_lift)
    a_lift_list.append(a_lift_norm)
    b_lift_list.append(b_lift_norm)
    c_lift_list.append(c_lift_norm)

print("==== Lifted objective values ====")
print(f"h^1(alpha_a_approx) = {hl_alpha_a_approx:.10f}")
print(f"h^1(beta_b_approx) = {hl_beta_b_approx:.10f}")
print(f"h^1(gamma_c_approx) = {hl_gamma_c_approx:.10f}")
print(f"h^1(w_hat_approx) = {hl_alpha_a_approx:.10f}")
print(f"h^1(w_escape_approx) = {hl_w_escape_approx:.10f}")
print(f"h(w^(t+1)) = {hl_w_t_approx:.10f}")
print(f"||alpha*a||={a_lift_list[-1]:.6f}, "
      f"||beta*b||={b_lift_list[-1]:.6f}, "
      f"||gamma*c||={c_lift_list[-1]:.6f}")

# Collapse selection (relaxed dominance)
collapse_ok = False
collapse_mode = None
X_check = None
if hl_w_t_approx < hl_alpha_a_approx:
    if (c_lift_norm > b_lift_norm) and (c_lift_norm > a_lift_norm):
        collapse_ok = True
        collapse_mode = "gamma"

```

```
100%|██████████████████████████████████████████████████████████████████████████████| 150000/150000 [01:02<00:00, 2414.98it/s]
```

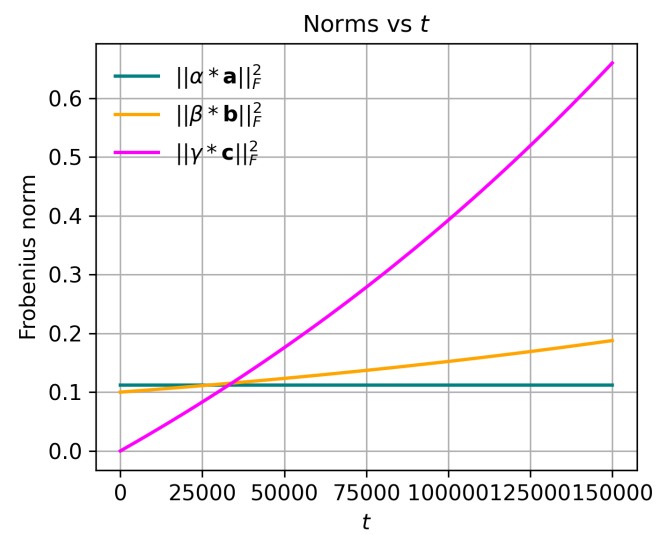
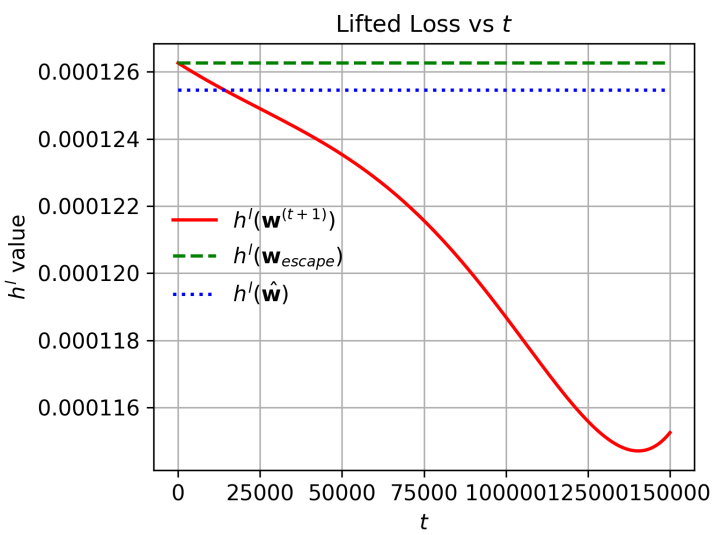
$$||\alpha \cdot a|| = 0.111978, \quad ||\beta \cdot b|| = 0.187710, \quad ||\gamma \cdot c|| = 0.659834$$

```
In [10]: # Set up a 1x2 grid of subplots
fig2, axes2 = plt.subplots(1, 2, figsize=(10, 4))

ax2D = axes2[0]
ax2D.plot(range(T), hl_w_list, color='red', label=r'$h^1(\mathbf{w}^{(t+1)})$', linewidth=2,
          linestyle='--', color='green', label=r'$h^1(\mathbf{w}_{\text{escape}})$', linewidth=2,
          linestyle=':', color='blue', label=r'$h^1(\hat{\mathbf{w}})$', linewidth=1.7)
ax2D.hlines(hl_alpha_a_approx, xmin=0, xmax=args.t,
            linestyle='--', color='green', label=r'$h^1(\mathbf{w}_{\text{escape}})$', linewidth=2,
            linestyle=':', color='blue', label=r'$h^1(\hat{\mathbf{w}})$', linewidth=1.7)
ax2D.set_xlabel(r'$t$', fontsize=11)
ax2D.set_ylabel(r'$h^1$ value', fontsize=11)
ax2D.tick_params(axis='both', labelsiz=11)
ax2D.set_title('Lifted Loss vs $t$', fontsize=12)
ax2D.legend(frameon=False, loc='center left', fontsize=11)
ax2D.grid(True)

ax2E = axes2[1]
ax2E.plot(range(T), a_lift_list, color='teal', label=r'$||\alpha * \mathbf{a}||_F^2$', 1
ax2E.plot(range(T), b_lift_list, color='orange', label=r'$||\beta * \mathbf{b}||_F^2$', 1
ax2E.plot(range(T), c_lift_list, color='magenta', label=r'$||\gamma * \mathbf{c}||_F^2$', 1
ax2E.set_xlabel(r'$t$', fontsize=11)
ax2E.set_ylabel('Frobenius norm', fontsize=11)
ax2E.tick_params(axis='both', labelsiz=11)
ax2E.set_title('Norms vs $t$', fontsize=12)
ax2E.legend(frameon=False, fontsize=11)
ax2E.grid(True)

plt.tight_layout()
plt.savefig('exp3_fig1.pdf', dpi=300, format='pdf')
plt.show()
```



Collapse from Tensor Space via Closed-Form Solution

- If collapsing c_lift term:

$$\check{X} = -(-\tilde{\gamma}^{(t)})^{\frac{1}{t}} \cdot \mathcal{A}^* \mathcal{A}(u_n v_r^\top + v_r u_n^\top) \hat{X} \in \mathbb{R}^{n \times r};$$

- If collapsing b_lift term:

$$\check{X} = (\tilde{\beta}^{(t)})^{\frac{1}{t}} \cdot (u_n q_r^\top) \in \mathbb{R}^{n \times r}.$$

As a result, $\check{X}$ lies in matrix space and represents a point that successfully escapes the attraction basin of $\hat{X}$.

Note: Since $\check{X}$ admits a closed-form expression, **there is no need to perform actual tensor lifting** — the procedure is purely simulated.

```
In [11]: print("collapse_mode is", collapse_mode)
if collapse_ok:
    if collapse_mode == "gamma":
        scale = np.sign(gamma_t) * (abs(gamma_t) ** (1.0 / 1)) if gamma_t != 0 else 0.0
        X_check = (scale * c_vec).reshape(n, r)
    elif collapse_mode == "beta":
        scale = np.sign(beta_t) * (abs(beta_t) ** (1.0 / 1)) if beta_t != 0 else 0.0
        X_check = (scale * b_vec).reshape(n, r)
```

collapse_mode is gamma

Run GD in Matrix Space (Post-Escape Phase)

```
In [12]: # If collapse performed, run GD from X_check
traj_after = []
losses_after = []
gradnorms_after = []
dists_after = []
#losses_full = list(losses_before)
#gradnorms_full = list(gradnorms_before)
#dists_full = list(dists_before)

if X_check is not None:
    traj_after, losses_after, gradnorms_after, dists_after = run_gd(
        A, Z, b, X_check, args.gd_lr, args.gd_steps_after_collapse, args.gd_gradtol)
    #losses_full.extend(losses_after)
```

```

#gradnorms_full.extend(gradnorms_after)
#dists_full.extend(dists_after)

print(len(traj_after))
print(len(losses_after))
print(len(gradnorms_after))
print(len(dists_after))

```

```

101
101
101
101

```

Full Procedure Visualization

```

In [13]: # Set up a 1x3 grid of subplots
fig3, axes3 = plt.subplots(1, 3, figsize=(10, 4))

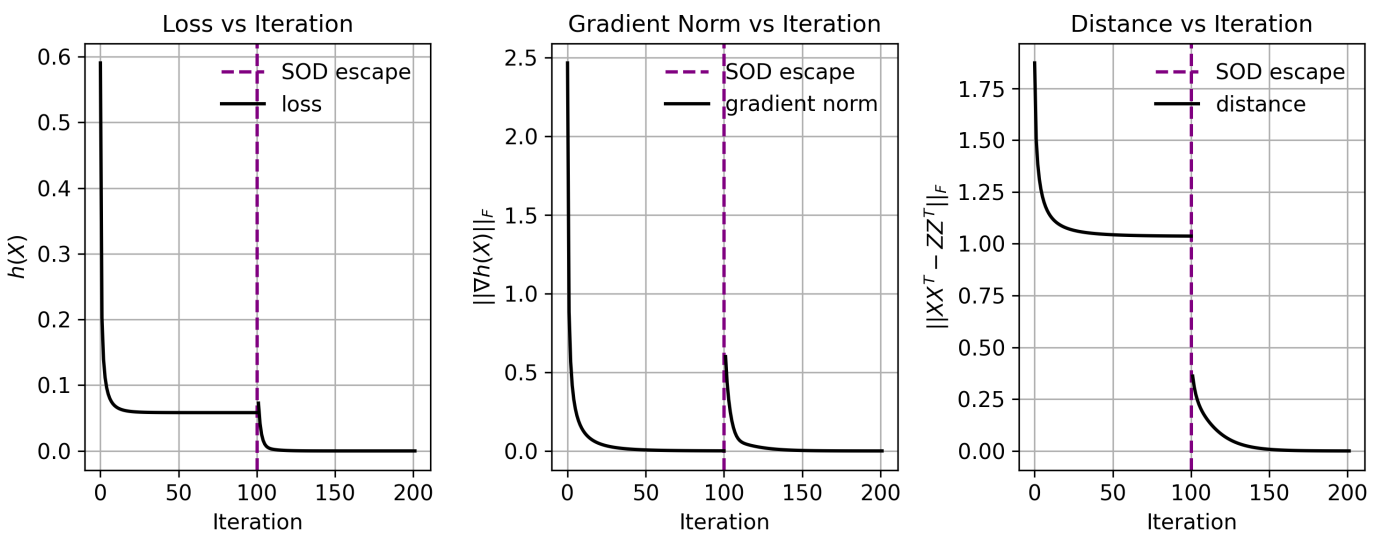
ax3A = axes3[0]
ax3A.axvline(len(losses_before) - 1, linestyle='--', color='purple', label='SOD escape',
ax3A.plot(range(args.gd_steps_before_lift + 1),
          losses_before, color='black', label='loss', linewidth=1.7)
if len(losses_after) > 0:
    ax3A.plot(range(args.gd_steps_before_lift + 1, args.gd_steps_before_lift + args.gd_s
              losses_after, color='black', linewidth=1.7)
ax3A.set_xlabel('Iteration', fontsize=11)
ax3A.set_ylabel(r'$h(X)$', fontsize=11)
ax3A.tick_params(axis='both', labelsize=11)
ax3A.set_title('Loss vs Iteration', fontsize=12)
ax3A.legend(frameon=False, fontsize=11)
ax3A.grid(True)

ax3B = axes3[1]
ax3B.axvline(len(gradnorms_before) - 1, linestyle='--', color='purple', label='SOD escap
ax3B.plot(range(args.gd_steps_before_lift + 1),
          gradnorms_before, color='black', label='gradient norm', linewidth=1.7)
if len(gradnorms_after) > 0:
    ax3B.plot(range(args.gd_steps_before_lift + 1, args.gd_steps_before_lift + args.gd_s
              gradnorms_after, color='black', linewidth=1.7)
ax3B.set_xlabel('Iteration', fontsize=11)
ax3B.set_ylabel(r'$||\nabla h(X)||_F$', fontsize=11)
ax3B.tick_params(axis='both', labelsize=11)
ax3B.set_title('Gradient Norm vs Iteration', fontsize=12)
ax3B.legend(frameon=False, loc='upper right', fontsize=11)
ax3B.grid(True)

ax3C = axes3[2]
ax3C.axvline(len(dists_before) - 1, linestyle='--', color='purple', label='SOD escape',
ax3C.plot(range(args.gd_steps_before_lift + 1),
          dists_before, color='black', label='distance', linewidth=1.7)
if len(dists_after) > 0:
    ax3C.plot(range(args.gd_steps_before_lift + 1, args.gd_steps_before_lift + args.gd_s
              dists_after, color='black', linewidth=1.7)
ax3C.set_xlabel('Iteration', fontsize=11)
ax3C.set_ylabel(r'$||XX^T - ZZ^T||_F$', fontsize=11)
ax3C.tick_params(axis='both', labelsize=11)
ax3C.set_title('Distance vs Iteration', fontsize=12)
ax3C.legend(frameon=False, fontsize=11)
ax3C.grid(True)

plt.tight_layout()
plt.savefig('exp3_fig2.pdf', dpi=300, format='pdf')
plt.show()

```



```
In [14]: # Plot 3d Trajectory
fig4 = plt.figure()
ax3d = fig4.add_subplot(1, 1, 1, projection='3d')
P_before = np.array([X.reshape(-1) for X in traj_before])
ax3d.plot(P_before[:, 0], P_before[:, 1], P_before[:, 2],
         color='blue', label='GD before SOD escape', linewidth=1.7)
if len(traj_after) > 0:
    P_after = np.array([X.reshape(-1) for X in traj_after])
    ax3d.plot(P_after[:, 0], P_after[:, 1], P_after[:, 2],
             color='green', label='GD after SOD escape', linewidth=1.7)
# markers with requested colors
ax3d.scatter(X_local[0, 0], X_local[1, 0], X_local[2, 0],
            color='blue', marker='o', s=50, label=r'local minimum  $\hat{X}$ ')
ax3d.scatter(Z[0, 0], Z[1, 0], Z[2, 0],
            color='red', marker='*', s=80, label=r'ground truth  $M^*$ ')
if X_check is not None:
    ax3d.scatter(X_check[0, 0], X_check[1, 0], X_check[2, 0],
                color='green', marker='o', s=60, label=r'escaped point  $\check{X}$ ')
# jump arrow
start = X_local.reshape(-1); end = X_check.reshape(-1); diff = end - start
if float(np.linalg.norm(diff)) > 1e-12:
    ax3d.quiver(start[0], start[1], start[2], diff[0], diff[1], diff[2], arrow_length=
                color='purple', linewidth=1.7, label='SOD escape direction')
ax3d.set_xlabel(r'$x_1$', fontsize=11)
ax3d.set_ylabel(r'$x_2$', fontsize=11)
ax3d.set_zlabel(r'$x_3$', fontsize=11)
ax3d.tick_params(axis='x', labelsz=11)
ax3d.tick_params(axis='y', labelsz=11)
ax3d.tick_params(axis='z', labelsz=11)
ax3d.set_title('Trajectory in 3D Space (with SOD escape)', fontsize=12)
ax3d.legend(frameon=False, ncol=3, loc='upper center', fontsize=11)
ax3d.view_init(elev=21, azim=102)

plt.tight_layout()
plt.savefig('exp3_fig3.pdf', dpi=300, format='pdf')
plt.show()
```

Trajectory in 3D Space (with SOD escape)

- GD before SOD escape
- GD after SOD escape
- local minimum $\hat{X}$
- ground truth M^*
- escaped point $\check{X}$
- SOD escape direction

